

← Go Back

Hardware

Portenta H7 Lite Connected ▾

Tutorials

- BLE Connectivity on Portenta H7
- Dual Core Processing
- Creating a Flash-Optimized Key-Value Store
- Getting Started with OpenMV and MicroPython
- Debugging with Lauterbach TRACE32 GDB Front-End Debugger
- Over-The-Air (OTA) Updates with the Arduino Portenta H7**
- Reading and Writing Flash Memory
- Secure Boot on Portenta H7
- Setting Up Portenta H7 For Arduino
- Voice Commands With The Arduino Speech Recognition Engine
- Portenta H7 as a USB Host
- Portenta H7 as a Wi-Fi Access Point

Home / Hardware / Portenta H7 Lite Connected / **Over-The-Air (OTA) Updates with the Arduino Portenta H7**

Over-The-Air (OTA) Updates with the Arduino Portenta H7

Learn how to perform an OTA update of the firmware on the Arduino Portenta H7

Author · Taddy Chung, José Bagur · Last revision · 01/31/2024

ON THIS PAGE

Overview

- Goals
- Hardware and Software Needed
- What OTA Means
- Instructions +
- Testing the Code
- Conclusion +
- Full Sketches +
- Troubleshooting

Overview

In this tutorial, you will learn how to use and allow firmware updates via **OTA (Over-The-Air)** feature with the **Arduino Portenta H7**. With this tutorial, you will be able to create a binary file to be used with the OTA feature and use the internal **QSPI** or a external **SD card** to accomplish the OTA (Over-The-Air) process.



To proceed with OTA using a SD Card, you will need to use a carrier or shield with a SD card slot, e.g Portenta Breakout, Portenta Max Carrier, Portenta Vision Shield.

Goals

The goals of this tutorial are:

- Create an OTA file required to use the OTA (Over-The-Air) feature.

- Use QSPI or SD card storage to load the firmware downloaded using the OTA feature.

Hardware and Software Needed

Arduino Portenta H7

Operative System: Linux or MacOS system, this procedure is not 100%

Arduino IDE 1.8.10+ or
Arduino Pro IDE 0.0.4+

USB-C® type cable (either
USB-A to USB-C® or USB-
C® to USB-C®)

Arduino IoT Cloud and
Arduino_Portenta_OTA
libraries

SD card (optional, you can
use QSPI instead)

Carrier or shield
compatible with the
Portenta H7 with a SD Card
slot, in case you choose to
use the SD Card.

What OTA Means

OTA (Over-The-Air) is a method of distributing wirelessly to end devices to update their firmware, configuration or security-related protocols. The purpose of this method is to change a device's behavior or its settings for better performance, for adding new features or to change its targeted usage.

Instructions

We will explain briefly the steps required to be able to use OTA (Over-The-Air) process with Arduino Portenta H7. It will consist of firmware OTA file creation and use of preferred storage mode (QSPI or SD card).

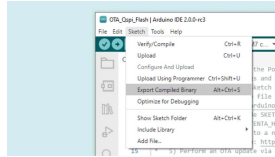
Firmware OTA File Creation

You will need to create the binary

The-Air) process to be able use it with either storage option stated previously. For the purpose of this tutorial, you will have to use the following script to create the binary file.

```
1  /*
2   This sketch can be
3   It needs to be used
4   - 'OTA_Qspi_Flash.
5   OR
6   - 'SD_Qspi_Flash.i
7
8   Steps to test OTA o
9   1) Upload this sket
10  2) In the IDE selec
11  3) Upload the expor
12  4) Choose a storage
13     eventually updat
14  5) Upload the sketc
15  */
16
17  void setLed(int blue
18    digitalWrite(LEDDB,
19    digitalWrite(LEDG,
20    digitalWrite(LEDRL,
21  }
22
23
24  void setup()
25  {
26    pinMode(LEDDB, OUTP
27    pinMode(LEDG, OUTP
28    pinMode(LEDRL, OUTP
```

This script will light up the RGB LED with 3 different colors in sequence. This code will need to be uploaded to the Arduino Portenta H7 firsthand. This is to verify whether the sketch compiles and is working correctly. After verifying this, in Arduino IDE, you will search for **Sketch > Export Compiled**



Exporting Binary for the Sketch

With the binary file ready, you can now create the OTA file needed to enable Over-The-Air process.

To continue, macOS or Linux environment is required. For Windows environment, it is possible to use virtualization software such as [Oracle VM Virtualbox](#) using your preferred choice of compatible Linux distribution. If you're not familiar with Linux environments, [Ubuntu](#) is one of many distributions that can help you explore Linux environment with ease of access.

Once you're comfortable with an environment, you will need a tool which can be found at the following link.



Arduino IoT Cloud Library
- Over-The-Air Tools

You will have to extract the library at a preferred location to be able to use the tools. Then, you will need to run on the terminal the following commands in sequence to be able to create the OTA file.

Copy the binary file into the library tool's folder

```
1 // Exported binary form
2 cp OTA_Usage_Portenta.?
```

Go inside that directory

```
1 cd ~/Arduino/libraries
```

Encode your binary file into

```
OTA_Usage_Portenta.ino.PORT
ENTA_H7_M7.lzss
```

```
1 // Argument format: ./
2 ./lzss.py --encode OTA_
```

Convert your encoded file into

`.ota` format

```
1 // Argument format: ./
2 ./bin2ota.py PORTENTA_
```

You can use

```
OTA_Usage_Portenta.ino.PORT
ENTA_H7_M7
```

as a sketch name for facilitated identification of the file. After this, you will have the `.ota` file of the sketch that you will use with the OTA process.

Installing Python 3 On Linux

If you are using Linux, maybe

need to install **Python 3** and the necessary modules. To do it execute the next command on your **Linux terminal**:

```
1 sudo apt install python3
```

You will also need to install the **crccheck** module on python by following the next instructions:

Installing pip on python:

```
1 //Necessary to install pip
2 sudo apt install python3-pip
```

Installing the crccheck necessary module on python:

```
1 //Necessary to run the script
2 pip install crccheck
```

Once you have done it, you should be able to run the bin2ota.py script successfully.

Uploading OTA file to the net

Now you can upload your .OTA file to an online reachable location, e.g.

OTA_Usage_Portenta.ino.PORTENTA_H7_M7.ota has been uploaded to:

https://downloads.arduino.cc/ota/OTA_Usage_Portenta.ino.PORTENTA_H7_M7.ota

You can change the default file location on the code by modifying the next line on the **"OTA_Qspi_Flash"** sketch or in the **"OTA_SD_Portenta"** sketch depending on which method are you going to follow:

```
1 static char const OTA_
```

It is important to know that if your OTA file is uploaded to an HTTPS website you will need to modify the next line in the code:

```
1 int const ota_download
```

This line is in **line 87** for the **"OTA_Qspi_Flash"** sketch or in **line 88** on the **"OTA_SD_Portenta"** sketch.

If you are going to use the example OTA file used in this tutorial you don't need to follow the steps in this section, just execute the sketch with the default file location.

Now you have two options to choose, use QSPI or use an SD Card to storage your

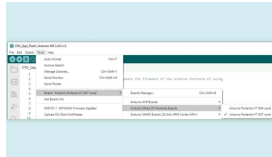


OTA file. You can use the left side index to jump to the option that you may

QSPI Storage Mode

Setting Up

To use internal **QSPI** storage for downloading the binary file via OTA (Over-The-Air), you will only need the Arduino Portenta H7 board connected to the computer with the **Arduino IDE**. With it, you will need to have selected the **Arduino Portenta H7 (M7 Core)** with the Flash split of **1 MB M7 + 1 MB M4** for the purpose of this tutorial and the corresponding port.



Arduino Portenta H7
Board Connection

Writing the Script

To proceed with a OTA using the QSPI flash, you can open the sketch from **Examples**
>**Arduino_Portenta_OTA** >
OTA_Qspi_Flash.

Do not forget to fill your
Wi-Fi AP SSID and
password on the
`arduino_secrets.h`
tab.

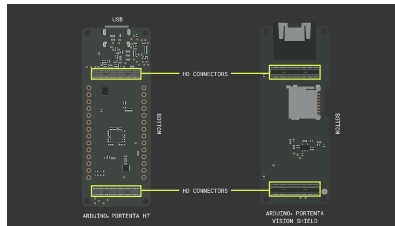
This sketch will connect to your Wi-Fi, verify whether the OTA feature is available by checking the installed firmware on your

Then prepare the OTA storage, download the .ota file from the internet, decompress it, reset the board so after the reboot it will swiipe the new firmware and you will notice it by checking the Built-in LED flashing on red, green and blue.

SD Card Storage Mode

Setting Up

To use the **SD card** as the preferred OTA (Over-The-Air) storage device, you can use the Arduino Portenta Vision Shield and Portenta H7 connected via **HD (High-Density)** Connectors.



With this, you will need the Arduino Portenta H7 board connected to the computer with the Arduino IDE. You will need to have selected the **Arduino Portenta H7 (M7 Core)** with the Flash split of **1MB M7 + 1 MB M4** for the purpose of this tutorial and the corresponding port.

Writing the Script

As same as QSPI storage mode, to proceed with a OTA using the SD Card, you can open the sketch from **Examples >**

Arduino_Portenta_OTA >

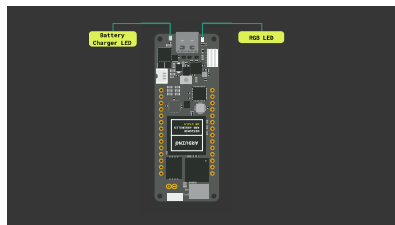
Do not forget to fill your Wi-Fi AP SSID and password on the `arduino_secrets.h` tab.

This sketch will connect to your Wi-Fi, verify whether the OTA feature is available by checking the installed firmware on your Portenta.

Then prepare the OTA storage, download the .ota file from the internet, decompress it, reset the board so after the reboot it will swipe the new firmware and you will notice it by checking the Built-in LED flashing on red, green and blue.

Testing the Code

Having successfully uploaded the code and completed the OTA process, you will be able to see the Arduino Portenta H7 blinking LED Red, Blue, Green in a cyclic manner. You will also be able to see process log in the Serial Monitor provided by Arduino IDE for more details.



[illegible]

Conclusion

We have now learned how to

You have now learned how to use the OTA feature provided by the Arduino Portenta H7, by updating its firmware. You will now be able to create the OTA file with the sketch designed by yourself and use this to update Arduino Portenta H7's firmware via OTA (Over-The-Air) feature with either QSPI or SD card storage mechanism.

Now, with the OTA capability in place, you can design future-proof system based on Arduino Portenta H7 or try different system disciplines using the OTA capability.

The complete script is as follows.

```
1  #include <Arduino_Po
2  #include <WiFi.h>
3  #include "arduino_se
4
5  static char const SS
6  static char const PA
7
8  static char const OT
9
10 void setup()
11 {
12     Serial.begin(11520
13     while (!Serial) {}
14
15     if (WiFi.status())
16     {
17         Serial.println("
18         return;
19     }
20
21     int status = WL_ID
22     while (status != W
23     {
24         Serial.print ("
25         Serial.print (S
26         Serial.println("
27         status = WiFi.be
28         delay(10000);
```

SD Card Storage Mode

```
1  #include <Arduino_Po
2  #include <WiFi.h>
3  #include "arduino_se
4
5  static char const SS
6  static char const PA
7
8  static char const OT
9
10 void setup()
11 {
12     Serial.begin(11520
13     while (!Serial) {}
14
15     if (WiFi.status())
16     {
17         Serial.println("
18         return;
19     }
20
21     int status = WL_ID
22     while (status != W
23     {
24         Serial.print ("
25         Serial.print (S
26         Serial.println("
27         status = WiFi.be
28         delay(10000);
```

Troubleshooting

For troubleshooting the issues that might have arose following the tutorial, you can use following tips to solve the issue.

If there has been an issue with Wi-Fi module, it means the device may have suffered from losing the Wi-Fi firmware partition. To solve this, you will have to use **PortentaWiFiFirmwareupdater** sketch found on Arduino IDE examples to fix the issue.

QSPI storage may throw

can use this guide of [Reading and Writing Flash Memory](#) in the section **Programming the QSPI Flash**. At this point, run the example and it should have been solved by eliminating error -3 (OTA Storage initialization error).

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

